



Concepto

Un **esquema** es un **contenedor** de objetos de base de datos (tablas, vistas, funciones, etc.).

➤ Permite organizar y administrar los objetos de forma lógica.



Cada usuario puede tener su propio esquema



Características

Organización lógica



Seguridad a nivel



Administración sencilla



Evita conflictos de nombres



Permisos asignados por esquema



Beneficios



Seguridad y control de acceso



Organización de objetos



Facilita trabajo en equipos



Evita conflictos de nombres



Separa funciones y objetos entre usuarios



Base de datos



dbo

predeterminado



Ventas

nuevo



Recursos Humanos



Contabilidad

→ Cada esquema agrupa objetos relacionados.



Ejemplo

crear esquema

1 Crear esquema **Ventas**

```
CREATE SCHEMA Ventas;
```

✓ El esquema **Ventas** se crea dentro de la base de datos Empresa.

2 Usar el esquema

```
CREATE TABLE Ventas.Clientes (  
  IdCliente INT PRIMARY KEY,  
  Nombre VARCHAR(100),  
  Ciudad VARCHAR(100)  
);
```

→ La tabla **Clientes** pertenece al esquema **Ventas**.



Recuerda

El esquema **dbo** es el predeterminado en SQL Server y pertenece al propietario de la base de datos (generalmente el usuario).

dbo

propietario



Esquema = contenedor

tablas · vistas · funciones · procedimientos





FUNCIONES

SQL



¿Qué son?

Bloques de código que realizan una tarea específica y devuelven un valor.

→ Reutilización

→ Lógica encapsulada



Escalar único valor

```
CREATE FUNCTION nombre (@param
tipo)

RETURNS tipo

AS

BEGIN

    -- instrucciones

    RETURN valor

END;
```

Ejemplo: IVA (18%)

```
CREATE FUNCTION fn_IVA (@monto DECIMAL(10,2))
RETURNS DECIMAL(10,2)
AS
BEGIN
    RETURN @monto * 0.18
END;
```

SELECT dbo.fn_IVA(1000) AS IVA;

 Resultado

IVA

180.00



Tabla conjunto de filas

```
CREATE FUNCTION nombre()

RETURNS TABLE

AS

RETURN

(

    -- consulta que devuelve
    filas

);
```



Ejemplo: Empleados con salario > 3000

```
CREATE FUNCTION fn_EmpleadosAltos()
RETURNS TABLE
AS
RETURN
(
    SELECT IdEmpleado, Nombre, Salario
    FROM Empleados
    WHERE Salario > 3000
);
```

SELECT * FROM dbo.fn_EmpleadosAltos();

 Resultado

IdEmpleado	Nombre	Salario
2	Luis Gómez	4500.00
3	Ana Pérez	5200.00



RECUERDA

Las funciones **no modifican datos** directamente. Devuelven un valor escalar o un conjunto de filas.



Usos comunes



Reutilización de código



Encapsular lógica de negocio



Mejorar legibilidad



Facilitar cálculos



Reducir duplicación de consultas complejas



Las funciones devuelven valores, no modifican el estado de la base.



Escalar → un solo valor



Tabla → conjunto de filas

ÍNDICES

SQL

¿Qué es?

Un **índice** es una estructura auxiliar que mejora la velocidad de búsqueda y recuperación de datos en una tabla.

Almacena valores de columnas en un orden específico con referencias a las filas.

TABLA ÍNDICE

Beneficios vs Desventajas

Mejora
velocidad de
búsqueda

Ocupa espacio adicional

Optimiza
JOIN,
WHERE,
ORDER BY

Lentas operaciones
INSERT/UPDATE/DELETE

Reduce tiempo de respuesta en aplicaciones

Mejores prácticas

Índices en búsquedas
frecuentes

Evita índices
innecesarios

Revisa y elimina no
utilizados

Monitorea rendimiento
y ajusta

Índices estratégicos = mejor rendimiento

Sintaxis básica

```
CREATE INDEX nombre_indice
ON tabla (columna1, columna2)

[UNIQUE] [CLUSTERED |
NONCLUSTERED];
```

UNIQUE valores únicos

CLUSTERED orden físico

NONCLUSTERED estructura separada

Ejemplo práctico

Tabla **Empleados**

IdEmpleado	Nombre	Departamento	Salario
1	Ana Pérez	Ventas	3500.00
2	Luis Gómez	Sistemas	4200.00
3	María Ruiz	Ventas	3800.00
4	Juan Soto	Sistemas	4100.00

```
CREATE INDEX Idx_Departamento ON Empleados
(Departamento); NONCLUSTERED
```

```
CREATE UNIQUE INDEX Idx_Empleado ON Empleados
(IdEmpleado);
```

```
CREATE CLUSTERED INDEX Idx_Salario ON Empleados
(Salario);
```

Índice CLUSTERED en Salario ordena físicamente los datos.

Recuerda

Los índices mejoran la velocidad de **lectura**, pero pueden afectar la velocidad de **escritura** y ocupan espacio adicional. Úsalos de forma estratégica.

lectura rápida

escritura más lenta

Equilibrio

velocidad de lectura vs. mantenimiento

Tipos

CLUSTERED

Ordena físicamente los datos. Solo uno por tabla.

NONCLUSTERED

Estructura separada con punteros a las filas. Varios por tabla.

UNIQUE

Asegura valores únicos. Puede ser CLUSTERED o NONCLUSTERED.

STORED PROCEDURES

SQL

¿Qué es?

Un procedimiento almacenado es un conjunto de instrucciones SQL que se almacena en la base de datos y se puede ejecutar cuando sea necesario.

Seguridad

Rendimiento

Reutilización

Beneficios

Seguridad: control de permisos

Rendimiento: plan de ejecución almacenado

Reutilización: código escrito una vez

Mantenimiento: lógica centralizada

Centraliza la lógica de negocio en la base de datos

Tipos de parámetros

ENTRADA (IN)

Envían valores al procedimiento. Ej: @IdDepartamento INT

SALIDA (OUT)

Devuelven un valor desde el procedimiento. Ej: @TotalEmpleados INT OUT

ENTRADA/SALIDA (INOUT)

Envían y devuelven un valor modificado. Ej: @Valor INT INOUT

Sintaxis básica

```
CREATE PROCEDURE
nombre_procedimiento

    @parametro1 tipo_dato,

    @parametro2 tipo_dato =
valor_default

AS

BEGIN

    -- instrucciones SQL

    SELECT, INSERT,
UPDATE, DELETE, etc.

END;
```

Los parámetros son opcionales y permiten enviar valores al procedimiento.

Ejemplo práctico

Empleados por departamento

```
CREATE PROCEDURE sp_EmpleadosPorDepto
@IdDepartamento INT
AS
BEGIN
    SELECT IdEmpleado, Nombre, Salario
    FROM Empleados
    WHERE IdDepartamento = @IdDepartamento;
END;
```

```
EXEC sp_EmpleadosPorDepto @IdDepartamento = 2;
```

```
EXEC sp_EmpleadosPorDepto 2;
```

IdEmpleado	Nombre	Salario
2	Luis Gómez	4200.00
4	María Ruiz	3800.00
7	Carlos López	4500.00

Devuelve los empleados del departamento 2

Valores de retorno

```
CREATE PROCEDURE sp_Empleados
AS
BEGIN
    -- instrucciones
    RETURN 0; -- éxito
END;
```

```
DECLARE @retorno INT;
EXEC @retorno = sp_Empleados;
```

RETURN devuelve un código de estado entero

Recuerda

Los **stored procedures** ayudan a **proteger los datos**, mejorar el **rendimiento** y **centralizar** la lógica de negocio dentro de la base de datos.

seguridad

rendimiento

reutilización

Características

Almacenados en la BD

Se ejecutan con EXEC

Parámetros IN / OUT / INOUT

Devuelven conjuntos de filas

Concepto tabla virtual

Una vista es una tabla virtual basada en el resultado de una consulta.

- No almacena datos físicamente, solo presenta información de una o varias tablas.

🔄 siempre actualizada

</> Sintaxis básica

```
CREATE VIEW nombre_vista AS

SELECT columna1, columna2, ...

FROM tabla

WHERE condición;
```

• CREATE VIEW

• SELECT

• FROM / WHERE

🔧 Ejemplo crear vista

```
CREATE VIEW Vista_Empleados AS
SELECT Nombre, Salario
FROM Empleados;
```

→ La vista muestra los datos como si fuera una tabla, pero no almacena información, solo consulta.

🛡️ Ventajas



Seguridad

Limita el acceso a ciertos datos sensibles.



Simplicidad

Simplifica consultas complejas en una sola vista.



Consistencia

Los datos siempre reflejan la información actual.



Dato clave: las vistas mejoran la organización y la seguridad.

📄 Usar la vista

2

SELECT * FROM Vista_Empleados;

Nombre	Salario
Ana Pérez	4500.00
Luis Gómez	5200.00
María Ruiz	4800.00

🔄 Vista_Empleados

→ resultado en vivo



La vista **Vista_Empleados** siempre refleja los datos actuales de la tabla **Empleados**.



RECUERDA

Las vistas facilitan el trabajo con datos, mejoran la seguridad y permiten presentar información de forma clara y organizada.